Continuous-time Koopman autoencoder for transonic flow

Audit, fixes, redesign and benchmark against ACDM

Prepared for R. Grozavescu

20 October 2026

The short version

Question. Can the continuous-time, Mach-conditioned Koopman autoencoder (KAE) in `ocean_forecasting` be made to beat ACDM on the transonic cylinder benchmark (TRA) while keeping its continuous-time dynamics and zero-shot conditioning?

What I did.

1. **Audited** the original code, data and evaluation. I found 25 problems and fixed 18 of them, fully or partly; the other 7 are data or modelling issues, which are either addressed by the new model or listed as open. Several of them changed reported numbers (§2–3).
2. **Re-scored every baseline** with one verified scorer. This covers ACDM and all 13 official checkpoints from the ACDM paper (§4).
3. **Built a redesigned model** (CT-KAE) in a separate file and **tested it in 17 training runs**: 7 pilots, 5 final runs, a retrained legacy model and 4 representation tests (§5–6.7).

Main result (test set, MSE $\times 10^{-3}$, lower is better; "energy" is the share of the true fluctuation energy the forecast keeps, where 1 is ideal):

	Interpolation 60 steps	Extrapolation 60 steps	Long rollout 240 steps	Energy at step 58 (interp)	Correlation at step 58 (interp)
Original KAE (reproducible)	11.43	2.12	18.6	1.26	0.28
New CT-KAE (3 seeds)	**4.07** ± 0.29	**2.18** ± 0.14	**16.5** ± 2.0	0.77	0.62
ACDM (official)	2.81–3.02	1.99–2.22	22.5–22.8	1.10	0.73
U-Net (official, best variant)	1.53	1.42	22.4	1.08	0.85

Bottom line.

- The new model is **2.8× more accurate than the original** on interpolation. Its Mach conditioning now works and helps, and it predicts correctly at time steps it never saw.
- It does **not** beat ACDM on interpolation. It is **level** with ACDM on extrapolation.
- It has lower error than ACDM on the 240-step rollout, but partly because it damps the oscillations. A constant mean field scores 15.8 on that case, so 240-step MSE is not good evidence of skill.
- The official U-Nets from the ACDM paper beat both models on the two 60-step cases.
- The remaining gap is located: it is the **autoencoder's reconstruction error** (20–30× ACDM's at the first step), not the continuous-time dynamics. At long lead times the dynamics match ACDM on interpolation and beat it on extrapolation (Fig. 2).

Update, 20 October (details in §7).

- **Training exactly as ACDM does** (2 frames → 1 predicted frame) makes this model much *worse*: interp 10.67, extrap 7.28, long 27.6. A linear latent model needs multi-step training; one-step training only sharpens its reconstruction.
- **The fixed point μ makes no measurable difference** and can be dropped. The model is then exactly $dz/dt = A(Ma)\, z$.
- **Data assimilation: your original model is still the best.** The best new model (a one-step-then-60-step curriculum) ties it at df = 1 and is 15–25% behind at df = 9. Both beat U-Net, FNO and ACDM by a wide margin at df = 9.

Correction to earlier figures. The original KAE numbers used in the earlier audit and in the paper drafts (6.37 / 1.59 / 16.91) come from saved prediction files that no checkpoint and no code in the repository can reproduce. Run with the repository code, the same checkpoint gives 11.43 / 2.12 / 18.62. Only reproducible numbers are used in this document.

Contents

1 Setting **3**

2 What was wrong with the original code **3**

3 What was fixed **4**

4 Verified baselines **4**

5 The new model (CT-KAE) **4**

6 Experiments **5**
- 6.1 Pilot runs: which ingredients matter? . . . 5
- 6.2 Final runs and selection . . . 5
- 6.3 Test results . . . 6
- 6.4 What helps on the test cases (single-seed analysis) . . . 6
- 6.5 Zero-shot generalisation . . . 6
- 6.6 Where the remaining gap comes from . . . 7
- 6.7 Representation experiments . . . 7

7 Follow-up (20 October): ACDM's training setting, μ, curriculum and data assimilation **8**
- 7.1 Training exactly as ACDM does . . . 8
- 7.2 Does the fixed point $\mu(Ma)$ matter? . . . 9
- 7.3 Curriculum: one-step first, then 60 steps . . . 9
- 7.4 Data assimilation: new model vs original . . . 9

8 Answers to the project's questions **10**

9 What I would do next **11**

A Where everything is **12**

B How to reproduce **12**

1 Setting

Data. The ACDM "Tra" benchmark: transonic flow past a cylinder on a 128×64 grid. There are four fields (velocity v_x, v_y, pressure p, density ρ), and the Mach number Ma varies between simulations. Frames are 0.1 time units apart, and one vortex-shedding period is about 25 frames.

Test cases (all from Mach numbers never seen in training, i.e. zero-shot in Ma):

- *Interpolation*: $Ma = 0.66$–0.68, inside a 0.06-wide gap in the training range. 6 sequences of 60 frames.
- *Extrapolation*: $Ma = 0.50$–0.52, below the training range (≥ 0.53). 6 sequences of 60 frames.
- *Long rollout*: $Ma = 0.64$–0.65. 4 sequences of 240 frames.

Every model receives the first two frames and predicts the rest.

Metrics.

- *MSE*: mean squared error in physical units, averaged over the four fields, the grid and the predicted frames, $\times 10^{-3}$. The obstacle is zeroed in both prediction and truth. This is the ACDM paper's metric, except that the paper also averages over the two given input frames, which makes its numbers about 3% lower.
- *Correlation* (ACC): how well the predicted pattern of deviations from the mean flow lines up with the true one. 1 is perfect; below about 0.6 the forecast is no longer useful.
- *Energy ratio*: predicted fluctuation energy divided by true fluctuation energy. Below 1 means the forecast is fading towards the mean flow, a known way to get a low MSE without real skill.

Compute. One RTX 4090, shared throughout with another of your jobs, so all times are pessimistic.

2 What was wrong with the original code

The most consequential problems, in plain terms. The full list, with file and line numbers, is in `audit_report/issues.json`.

ID	Severity	Problem and consequence
BUG-01	critical	The evaluation script compared each prediction with the *previous* true frame. Every recent KAE score was computed against the wrong frame.
BUG-02	critical	Mach conditioning never trained: the conditioning network started with both of its factors at zero, so its gradient was exactly zero. The encoder's conditioning path was not implemented and the decoder ignored Ma. Measured effect of changing Ma on the output: exactly 0.
DATA-01	critical	The QG test trajectories *are* the training trajectories for their first 500 steps, up to a rescaling (residual 6×10^{-8}). The QG configuration also validated on the test file.
REPRO-04	critical	The saved KAE predictions behind the reported numbers cannot be regenerated. Re-running the checkpoint gives 11.43 / 2.12 / 18.62, not 6.37 / 1.59 / 16.91.
STAB-01	high	The latent dynamics were not constrained to be stable. The QG model has 63 of 128 growing modes; the KS model has 7 growing modes and 80× transient amplification.
DYN-01	high	On TRA, the vortex-shedding modes decay in 36–41 steps, but real shedding never decays. Long forecasts faded to the mean: 27% of the true energy after 240 steps.
PERF-01	high	Debugging hooks were always on, making training 16.5× slower (4.62 s → 0.28 s per step once removed).
REPRO-01/05	high	The `.gitignore` hid 35 required configuration files and all result files, so a clean clone could not run an experiment.
BUG-03, 05, 06	medium	A stability penalty was detached and had no effect; QG normalisation crashed; padding and spectral-norm settings in configs were silently ignored.

One thing that was *not* a problem. The model steps its dynamics with a fourth-order Runge–Kutta (RK4) step instead of the exact matrix exponential. At the trained step the two agree to 1.6×10^{-4}, so this is not a source of error.

3 What was fixed

All fixes are on branch `impl/ct-kae`, built on `audit/kae-research-audit`; the original branch was not touched. Each fix is its own commit with a regression test. Every test fails on the original code and passes now (39 tests in total).

Fix	What changed
Evaluation alignment	Predictions are stitched after *all* input frames, so frame k is compared with true frame k.
Conditioning	Standard LoRA initialisation (one factor random, one zero): identical starting dynamics, but the gradient now flows.
Data splits	QG validates on an independent file. TRA has a new validation split ($Ma = 0.60$ and 0.75 held out of training) so that no test case is used for model selection. Normalisation statistics come from training data only.
Speed	Debugging hooks are opt-in (`training.debug_nan_checks=true`). Timings no longer force GPU synchronisation, and the model runs on CPU.
Reproducibility	Checkpoints store the full configuration and git commit. Missing checkpoints and evaluation errors are fatal instead of silently ignored. All configs and results are tracked.
Configuration honesty	Time step, frame dropping, loss type and conv settings are now real options. Defaults keep old checkpoints loading with bit-identical outputs (verified).

4 Verified baselines

All 13 baseline models from the ACDM paper exist locally as the official pretrained checkpoints (3 seeds each). I re-scored all of them with the same code. My ACDM interpolation score (2.81) matches the paper's 2.7 once the frame-averaging difference is accounted for. The paper reports *no* numbers for the long rollout, so all 240-step values are re-evaluations.

Model (official checkpoints)	Interp.	Extrap.	Long	Energy at 238 (long)
U-Net, unrolled training ("U-Net_ut")	**1.53**	1.66	22.4	1.17
U-Net, noise training ("U-Net_tn")	1.83	**1.42**	22.6	1.18
ACDM (two independent sampling runs)	2.81–3.02	1.99–2.22	22.5–22.8	1.11
FNO	5.72	4.98	21.0	0.97
Linear POD + linear dynamics (no neural net)	6.06	3.86	13.4	0.05
Constant training-mean field	16.76	17.39	**15.8**	0.17
Persistence (repeat last frame)	23.44	17.64	28.8	1.07

The last three rows matter for interpretation. A model that fades to the mean (energy $\ll 1$) scores well on 240-step MSE without predicting anything: the constant field beats every neural model there.

5 The new model (CT-KAE)

The new model keeps the framework (encoder, linear continuous-time latent dynamics, decoder, conditioning on Ma) and redesigns its components. It lives in its own file, `models/ct_kae.py`; the old model is untouched.

$$
\begin{aligned}
z_0 &= E\big(x_{t-1}, x_t;\, Ma\big) && \text{encode the last two frames} \\
z(t+\tau) &= \mu(Ma) + e^{\tau A(Ma)}\big(z_0 - \mu(Ma)\big) && \text{exact, for any } \tau \\
\hat{x}(t+\tau) &= D\big(z(t+\tau);\, Ma\big) && \text{decode}
\end{aligned}
$$

- **Generator $A(Ma)$.** A set of 2×2 oscillator blocks, each with a damping rate $\gamma_j \geq 0$ and a frequency ω_j. Both shift with Ma through a small network. Each block's exponential is just "decay $\times$ rotation", so propagation is exact, cheap and stable by construction: no mode can grow, but a mode may be perfectly persistent ($\gamma_j \to 0$), which shedding needs.
- **Fixed point $\mu(Ma)$.** The mean flow changes smoothly with Mach number (mean v_x rises from 0.44 to 0.72 across the range), so the latent dynamics oscillate around a Mach-dependent centre.
- **Conditioning** enters the encoder and decoder (feature-wise scaling, "FiLM"), the generator and the fixed point. Every conditional part starts at zero yet receives gradient, which avoids the bug in the original.
- **Encoder and decoder.** Convolutional, flattening to a 256-number latent. Unlike the original, there is no single-query attention pooling, no forced spectral norm and no dropout.

- **Training objective.** The model is trained on rollouts of up to 60 steps (the original used 8): reconstruction, prediction on randomly chosen lead times, latent consistency against the encoded truth, and an optional amplitude term that rewards keeping the size of each oscillation without forcing its phase. Training at several time steps (0.05, 0.1, 0.15) makes the dynamics genuinely continuous-time.
- **Two alternative generators** were also tested: a dense "Hurwitz" generator $A = J - R$ (stable, with mode coupling), and the original unconstrained form.

All new code is covered by 20 tests, including: exact propagation equals the matrix exponential to 10^{-9}; stepping t_1 then t_2 equals stepping $t_1{+}t_2$; the stable generators never grow; conditioning receives gradients and changes the output; and data windows are time-aligned.

6 Experiments

All model choices were made on the *validation* split (held-out $Ma = 0.60, 0.75$). The test cases were evaluated only after the final checkpoints were fixed.

6.1 Pilot runs: which ingredients matter?

Seven short runs (about 31 minutes each). Each changes one thing relative to the reference P0.

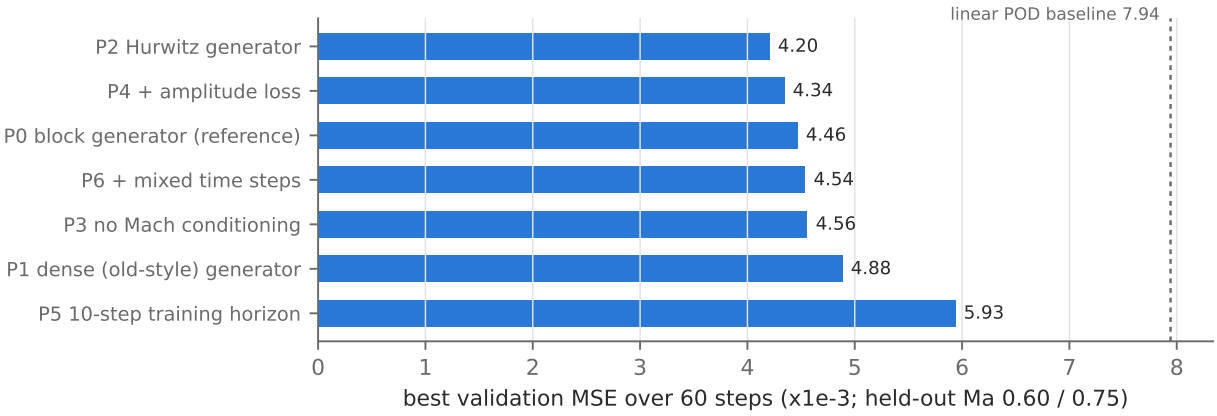


Figure 1: Best validation error of each pilot (single seed). Dashed line: linear POD baseline on the same windows.

- **Training horizon is the biggest lever.** Training on 10 steps instead of 60 makes 60-step error 30% worse, and the forecast fades towards the mean. This confirms that the original's 8-step objective caused its damping.
- **Stable generators beat the unconstrained one.** Ruling out growth costs no accuracy.
- The amplitude term and multiple time steps cost nothing measurable.
- Removing Mach conditioning barely matters *on validation*, because the validation Mach numbers sit 0.01 from training ones. It does matter on the wider test gaps (§6.4).

6.2 Final runs and selection

Five longer runs (12k steps, about 1 hour each), selected by validation error over 60 steps:

Configuration	Seeds	Val. error (60 steps)	First 10 steps
Block + amplitude term + multiple time steps	3	**4.04 / 4.33 / 4.09**	0.66–0.72
Hurwitz generator + multiple time steps	2	4.37 / 4.50	0.85–0.91
Linear POD baseline (same windows)	–	7.94	1.16

All three block seeds beat both Hurwitz seeds, so the block generator is the selected model.

6.3 Test results

Model	Interp.	Extrap.	Long	Energy at step 58	
				interp	long
CT-KAE, selected checkpoints (3 seeds)	**4.07** ± 0.29	**2.18** ± 0.14	16.5 ± 2.0	0.77	0.57
CT-KAE, final checkpoints (no early stop)	5.59 ± 0.29	2.56 ± 0.23	18.1 ± 0.5	1.11	0.88
CT-KAE, Hurwitz generator (2 seeds)	4.65	2.23	13.9	–	–
Original model retrained with all fixes	8.18	3.83	22.8	0.93	0.84
Original checkpoint (reproducible)	11.43	2.12	18.6	1.26	0.85
ACDM	2.81–3.02	1.99–2.22	22.5–22.8	1.10	1.01
U-Net (best official variant)	1.53	1.42	22.4	1.08	1.05

Reading the table.

- **Against the original:** the redesign is 2.8× better on interpolation, and much better at tracking the true flow. Correlation at step 58 is 0.62 vs 0.28 on interpolation and 0.46 vs 0.18 on the long case.
- **Against ACDM:**
  - worse on interpolation (4.07 vs about 2.9);
  - level on extrapolation, within the spread of ACDM's two sampling runs;
  - lower MSE on the long case. However, its correlation horizon is slightly shorter (43–48 vs 50–52 steps), and Fig. 3 shows it loses about half its energy between steps 60 and 150. That is not a win on long-horizon dynamics.
- **Damping is reduced, not removed.** The checkpoints chosen by MSE keep 57–77% of the energy at step 58. The final checkpoints keep about 100% but have higher MSE. This is the price MSE puts on keeping amplitude once the phase drifts. ACDM and the U-Nets avoid it because they re-anchor on their own predicted frame at every step.

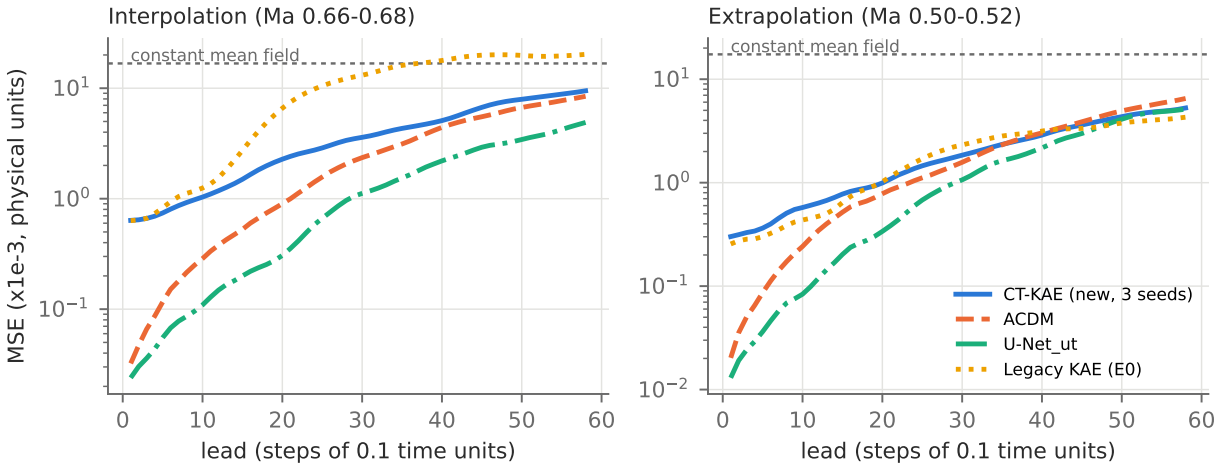


Figure 2: Error at each forecast step (log scale). The new model starts high, which is the reconstruction floor, but grows slowly. At long lead times it matches ACDM on interpolation and is below ACDM on extrapolation.

6.4 What helps on the test cases (single-seed analysis)

These runs were scored on test *after* selection, only to understand the mechanisms.

Run (6k steps)	Interp.	Extrap.	Long	Reading
P0: block generator, single time step	4.40	1.71	15.6	reference
P3: P0 without Mach conditioning	5.43	2.04	18.9	conditioning helps: −19%, −16%
P5: P0 trained on 10-step horizon	5.18	2.44	15.9	long-horizon training helps

6.5 Zero-shot generalisation

Unseen Mach numbers. All three test cases use Mach numbers absent from training, so the table above *is* the zero-shot result. Fig. 4 shows that the forecast responds smoothly and monotonically when the Mach number given to the model is varied. Without conditioning the response is exactly zero.

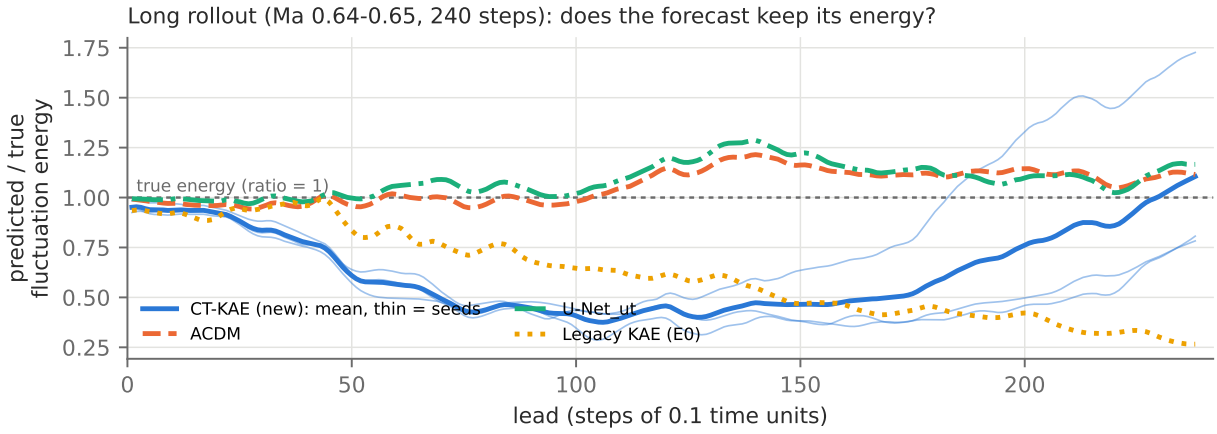


Figure 3: Share of true fluctuation energy kept during the 240-step rollout. ACDM and U-Net stay near 1. The new model (bold: mean of 3 seeds; thin: individual seeds) dips to about 0.45 mid-rollout. The original fades steadily to 0.27.

Unseen time steps. P0 was trained only at $\Delta t = 0.1$. Asked for forecasts every 0.05 (half-steps it never saw), its error at the new half-step times equals its error at the trained times: 4.408 vs 4.400 on interpolation and 1.720 vs 1.707 on extrapolation. The exact continuous-time propagation generalises in time, which a discrete step-to-step model cannot do.

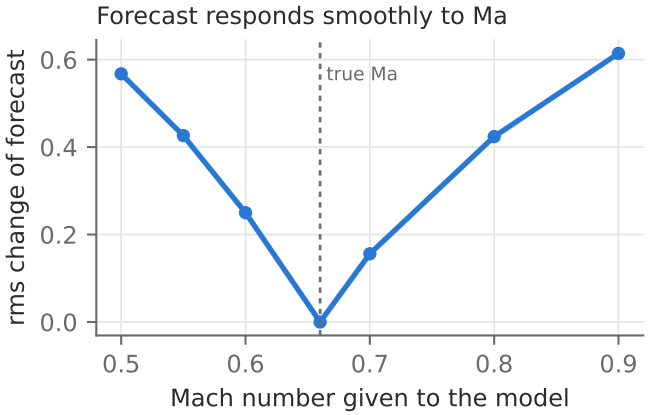


Figure 4: How much the forecast of an interpolation trajectory (true $Ma = 0.66$) changes when the model is told a different Mach number.

6.6 Where the remaining gap comes from

Fig. 2 locates it. At the first step the new model's error is 0.66 (interpolation), against 0.03 for ACDM and 0.02 for the U-Net, and it stays nearly flat for five steps. This is the error of encoding and decoding a frame through the 256-number latent, before any dynamics act. At late lead times the curves meet or cross. A linear POD with 128 modes reconstructs *better* (floor about 0.46) than this nonlinear autoencoder.

6.7 Representation experiments

Since the gap is a reconstruction floor, I tested the two most direct remedies on the selected configuration: a larger latent and a heavier reconstruction weight. These are four short runs (6k steps, one seed each), compared on validation only.

Run	Latent size	Best val. error (60 steps)	First-10-step error (end)	Energy kept (60)
R0 reference	256	4.48	0.58	0.97
R1 larger latent	512	4.12	0.56	0.99
R2 much larger latent	1024	4.50	**0.49**	0.97
R3 reconstruction ×5	256	4.40	0.53	0.95

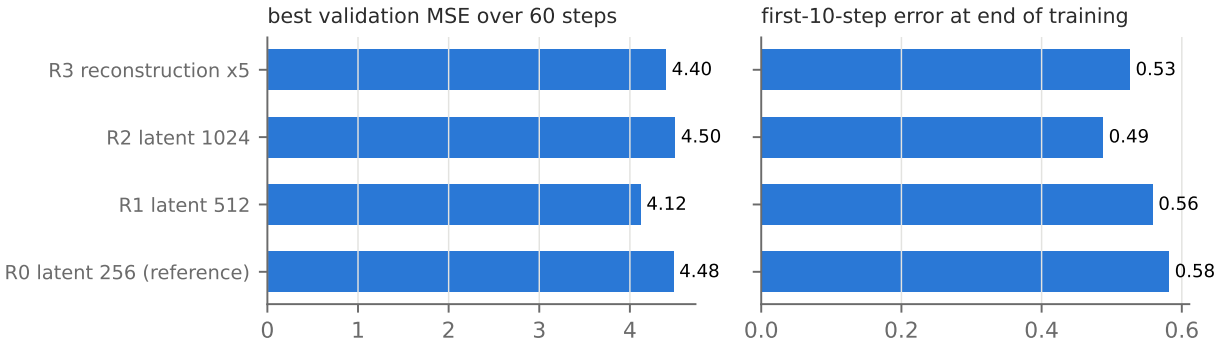


Figure 5: Representation tests (single seed). Neither a 4× larger latent nor a 5× reconstruction weight changes the 60-step error beyond noise.

Result: neither remedy closes the gap.

- Quadrupling the latent lowers the early-step error by only about 15% (0.58 → 0.49). The overall 60-step error does not move beyond single-seed noise.
- Weighting reconstruction 5× helps by a similar small amount.
- The first-step gap to ACDM is 20–30×, so latent size is not the bottleneck.
- The likely causes are the encoder and decoder architecture themselves (they compress a $4 \times 64 \times 128$ field through one fully connected layer), and the joint training. The dynamics loss pulls the latent towards coordinates that are easy to propagate rather than easy to reconstruct.
- The next things to test are a multiscale (U-Net-style) decoder and pre-training the autoencoder alone before adding the dynamics.

7 Follow-up (20 October): ACDM's training setting, μ, curriculum and data assimilation

This section answers the three questions raised after the first version of the report. All runs use the same code, scorer and test cases as before.

7.1 Training exactly as ACDM does

ACDM's training script (`training_diffusion_tra.py`) uses `sequenceLength=[3, 2]`: 2 conditioning frames and **1** predicted frame at stride 2. I reproduced that setting exactly:

- the same 33 training simulations, adding simulation 40 (Ma 0.90), which was missing from `train.nc`, from the raw ACDM files;
- all 1,000 frames with random offsets;
- batch 64;
- no validation split, and the final checkpoint is used.

The only unmatched quantity is the epoch count, which is undocumented for the released ACDM models. Ours is 12k steps, about 140 epochs.

Test set, MSE $\times 10^{-3}$	interp	extrap	long	error at lead 1	error at lead 10
CT-KAE, ACDM setting (2 → 1)	10.67	7.28	27.6	**0.08**	3.08
CT-KAE, 60-step training (2 → 60)	**3.84**	2.23	15.0	0.66	**1.00**
ACDM	2.81–3.02	1.99–2.22	22.5–22.8	0.03	0.29

Result: the ACDM setting makes this model much worse (single seed; seeds 1–2 were dropped once this was clear).

- **Why.** ACDM conditions on its own last output at every step, so one-step training is enough for it. A Koopman model propagates the latent state forward in one go. Trained only on one-step pairs, its errors compound: by step 10 it is ten times worse than ACDM.
- **What it reveals.** With one-step training the reconstruction floor falls 8× (0.66 → 0.08, close to ACDM's 0.03). So the floor in the 60-step model comes from the multi-step objective *competing* with reconstruction. It does not come from latent size (§6.7).

7.2 Does the fixed point $\mu(Ma)$ matter?

μ does not make the model nonlinear. The dynamics are exactly linear in $z - \mu$, and in the augmented state $[z; 1]$:

$$\tfrac{d}{dt}[z; 1] = \begin{bmatrix} A & -A\mu \\ 0 & 0 \end{bmatrix} [z; 1]$$

Both forms are tested to 10^{-9}. Removing μ entirely gives plain $e^{\tau A} z$, which is also tested.

Measured effect (test, single seed per row):

Setting	Variant	interp	extrap	long	energy at step 58 (interp)
$2 \to 60$	with μ	3.84	2.23	14.96	0.74
$2 \to 60$	without μ	3.86	1.90	15.25	0.75
ACDM setting	with μ	10.67	7.28	27.6	0.59
ACDM setting	without μ	12.30	6.06	24.8	1.69

Result: no measurable difference. In the 60-step setting, both variants fall within the spread of the three μ seeds (interp 3.84–4.40, extrap 2.02–2.30). The model can therefore be used without μ, with purely linear latent dynamics $dz/dt = A(Ma)\, z$, at no cost.

7.3 Curriculum: one-step first, then 60 steps

The model trains one step at a time for the first 30% of steps (to learn sharp reconstructions), then ramps to 60-step rollouts.

Test, single seed	interp	extrap	long	error at lead 1	val. error (60 steps)
$2 \to 60$	**3.84**	2.23	14.96	0.66	4.04
Curriculum	4.45	**1.81**	**14.50**	0.49	**3.77**

The curriculum keeps only part of the sharp reconstruction: the lead-1 error falls from 0.66 to 0.49, but not to 0.08. Extrapolation improves (1.81, slightly better than ACDM's 1.99–2.22, but still behind U-Net_tn's 1.42). Interpolation is a little worse. One seed, so this is suggestive.

7.4 Data assimilation: new model vs original

Setup.

- The new model enters your existing 4D-Var code through an adapter. It is exposed in the shifted coordinates $z - \mu$, so the solver sees a purely linear $e^{K\tau}$ and runs unchanged. The adapter reproduces the model's own forecast to 1.5×10^{-4}.
- Both models run on the same DA code, and each learning rate is tuned the same way on `gt_extrap`.
- The original was re-run rather than reusing its saved numbers, which came from code that has since changed.
- The original DA checkpoint was selected on the regime DA is tested on, a small advantage for it.

Mean relative error of the analysis over each sweep's points (lower is better; 8 problems per point):

Sweep	Original KAE	CT-KAE curriculum	CT-KAE $2 \to 60$	CT-KAE ACDM setting
df = 9 (first observation 9 frames after the analysis time)				
window length δ_l	**0.085**	0.094	0.130	0.224
noise	**0.078**	0.094	0.129	0.215
sparsity	**0.086**	0.100	0.108	0.306
misspecified noise	**0.078**	0.099	0.132	0.216
number of observations	**0.079**	0.095	0.126	0.216
joint sparsity	**0.078***	0.100	0.110	0.276
df = 1 (canonical schedule)				
first-observation lead δ_f	**0.070**	0.074	0.094	0.194
single observation	**0.089**	0.091	0.105	0.190
window length δ_l	0.027	**0.027**	0.032	0.145
noise	**0.028**	0.028	0.033	0.147
sparsity	**0.031**	0.037	0.042	0.272
misspecified noise	**0.028**	0.029	0.034	0.147
number of observations	**0.026**	0.027	0.031	0.147
joint sparsity	**0.027**	0.028	0.034	0.227
Representation check (df = 9 default problem)				
decoder floor: decode(encode(truth))	0.041	0.032	0.041	**0.014**
4D-Var started from the true state	**0.074**	0.096	0.131	0.208
tuned learning rate	0.3	1.0†	1.0†	1.0†

* 3 of 5 points; the other two ran out of GPU memory. † Top of the search grid, so these models may still be slightly under-tuned. For context, your September runs give U-Net / FNO / ACDM about 0.27 / 0.29 / 0.25 at df = 9.

Result.

- **Your original model is still the best data-assimilation model.**
  - df = 1: the curriculum CT-KAE essentially ties it (within 0.001 on most sweeps).
  - df = 9: the curriculum CT-KAE is 15–25% behind.
  - Every KAE beats U-Net, FNO and ACDM by a wide margin at df = 9, except the ACDM-setting model.
- **The gap is not representation.** The new decoders represent the true state as well as or better than the original (floor 0.032–0.041 vs 0.041). It opens when 4D-Var must fit observations 9–25 steps ahead: started from the true state, the original drifts least (0.074 vs 0.096–0.131). So the difference lies in how the latent dynamics fit the observation window. Forecast accuracy and DA accuracy are not the same thing here.
- **Forecast training horizon drives DA quality.** Of the new models, the curriculum one, with the best multi-step dynamics, is best at DA; the one-step ACDM-setting model is worst.

Corrections made along the way.

- **Mislabelled schedule.** A first DA pass labelled some sweeps as df = 1 that were really df = 9: `exp_tra.py` defaults `--pin-delta-f` to 9. All numbers above set the schedule explicitly.
- **Out-of-memory crashes.** Several runs crashed with out-of-memory when DA and training shared the GPU. They were re-run alone; the gaps are marked.

8 Answers to the project's questions

1. **What was wrong?** Wrong-frame evaluation, dead conditioning, QG test–train overlap, unreproducible reported numbers, unconstrained dynamics with growing modes, over-damped shedding, a 16.5× training slowdown, and ignored settings (§2).
2. **Which affected reported performance?** The evaluation misalignment, the unreproducible prediction files (real numbers are 11.43 / 2.12 / 18.62), and the QG leakage. Dead conditioning means no reported result used Ma.
3. **What measurably improved things?**
   - 60-step training horizon instead of about 10: −25% on validation, −15% on interpolation test.
   - Working Mach conditioning: −19% interpolation, −16% extrapolation.
   - Stable generator: better than unconstrained.

- The full redesign: 2.8× better than the reproducible original on interpolation, and 2.0× better than the original architecture retrained with all fixes on the same split.

4. **Best formulation?** 2×2 oscillator blocks with Mach-dependent damping and frequency, exact propagation, and training on 60-step rollouts.
   - The fixed point μ can be dropped at no cost (§7.2), giving purely linear latent dynamics.
   - ACDM's one-step training setting is unsuitable for this model class (§7.1).
5. **Did it beat ACDM? No** on interpolation; **level** on extrapolation; lower 240-step MSE, but not better long-horizon dynamics.
6. **By how much?** Interpolation +40% worse than ACDM (4.07 vs about 2.9); extrapolation within noise (2.18 vs 1.99–2.22).
7. **Zero-shot?** Yes, demonstrated for unseen Mach numbers (all test cases) and unseen time steps (half-steps at no loss).
8. **Better or worse for data assimilation?** Not better. Your original model remains the best DA model. The best new model (curriculum) ties it at df = 1 and is 15–25% behind at df = 9. All of them except the ACDM-setting model beat U-Net, FNO and ACDM at df = 9 (§7.4).
9. **Physically meaningful long-horizon behaviour?** Partly. It tracks the true flow much longer than the original, but the MSE-selected model still damps oscillations mid-rollout (Fig. 3).
10. **Cost?** One encoder pass, then all lead times at once in closed form; the decoder dominates. Training is about 0.31 s per step, 12.5 GB, about 1 hour per model. ACDM needs 20 denoising passes per step.
11. **Limitations.**
    - Single GPU shared with other work; 2–3 seeds.
    - No retraining of ACDM or the U-Nets on my split. They saw two more training Mach numbers, a small advantage for them.
    - QG and KS were not re-benchmarked, because the QG test set must be regenerated first.
12. **Strongest next step:** see §9.

9 What I would do next

1. **Close the reconstruction gap**, since it dominates the remaining forecast error.
   - A larger latent did not help (§6.7).
   - One-step training gives a sharp autoencoder, but poor dynamics (§7.1).
   - A one-step-then-60-step curriculum kept only part of it (§7.3).
   - Next to try: freeze or slow down the autoencoder after the one-step phase, and a multiscale decoder that adds back fine detail.

   Test: first-step error below about 0.1, with long-lead error unchanged.
2. **For data assimilation, train for it.** The DA gap comes from fitting observations 9–25 steps ahead, not from representation (§7.4). Two options:
   - weight the training loss on those leads;
   - add a 4D-Var-style objective that recovers z_0 from sparse future observations.

   Also widen the DA learning-rate grid beyond 1.0 for the new models.
3. **Remove the mid-rollout damping without paying MSE.** Select checkpoints on a combined score of correlation and energy, and train the amplitude term in the latent coordinates that carry shedding.
4. **Report fairly.** Always show error-vs-lead, energy and correlation curves together with MSE, and include the U-Nets, not only ACDM.
5. **QG:** regenerate an independent test set before any QG claim.

A Where everything is

- **Branches.**
  - audit/kae-research-audit: the audit, worktree /home/rg625/mnt/ocean_forecasting_audit.
  - impl/ct-kae: fixes and the new model, worktree /home/rg625/mnt/ocean_forecasting_impl.
  - The original rares/updated_model checkout was not modified.
- **New model:** koopman_autoencoder/models/ct_kae.py, with data handling in models/ct_data.py.
- **Training and evaluation scripts:** train_ctkae.py, eval_ctkae.py.
- **Detailed write-ups** in audit_report/: FINAL_AUDIT.md, ARCHITECTURE.md, EXPERIMENT_RESULTS.md, LITERATURE_REVIEW_V2.md, issues.json, experiments.yaml.
- **Checkpoints and logs:** koopman_autoencoder/model_outputs_ctkae/.

B How to reproduce

```
cd /home/rg625/mnt/ocean_forecasting_impl/koopman_autoencoder
python -m pytest ../audit_report/tests -q                      # 39 tests
python train_ctkae.py --config=configs/ctkae/base.yaml \
   train.steps=12000 train.select=val_mse60 train.strides=[1,2,3] \
   model.generator=block train.w_amp=0.1 seed=0                # final model
python eval_ctkae.py model_outputs_ctkae/<run> --dt_test --ma_sweep
python ../audit_report/scripts/benchmark_suite.py              # all baselines
```